

ROS2 TF2 실습 — 객체 탐지 연동 & Launch 통합

Dynamic TF Broadcaster · RViz2 시각화 · Launch 파일

cchyun@gmail.com

Part 1. Dynamic TF Broadcaster

— 객체 탐지 결과를 TF로 발행

- YOLO로 탐지한 객체를 TF 트리에 동적으로 추가하여 공간 위치 표현
 - 학습 내용
 - 구독한 이미지 객체 탐지 결과를 3D 공간 좌표로 매핑
 - `tf2_ros.TransformBroadcaster`로 동적 프레임 실시간 발행
 - `camera_link`를 부모로 하는 객체 프레임 생성
 - 사전 요구사항
 - `my_robot_interfaces` 패키지가 빌드되어 있어야 함 (08.02 참고)
 - `camera_pkg`의 `img_pub`, `yolo_pub` 노드가 준비되어 있어야 함 (07.03, 08.01 참고)

- 정적(Static) vs 동적(Dynamic) 브로드캐스터

구분	StaticTransformBroadcaster	TransformBroadcaster (동적)
토픽	<code>/tf_static</code>	<code>/tf</code>
발행 주기	단 한 번	주기적으로 반복
사용 예시	고정된 센서 마운트	움직이는 객체, 로봇 위치
데이터 유지	래치(Latched)	실시간 갱신 필요

- 이번 실습에서는 **친구(사람)의 위치** 를 TF로 계속 발행하여 RViz2에서 실시간 추적

1-1. tf_yolo_broadcaster.py 작성

src/tf_tutorial_pkg/tf_tutorial_pkg/tf_yolo_broadcaster.py

```
import rclpy
from rclpy.node import Node
from my_robot_interfaces.msg import ObjectDetectionArray
from geometry_msgs.msg import TransformStamped
import tf2_ros

class IfYoloBroadcaster(Node):
    def __init__(self):
        super().__init__('tf_yolo_broadcaster')
        self.br = tf2_ros.TransformBroadcaster(self)
        self.create_subscription(
            ObjectDetectionArray, '/image_yolo', self.callback, 10)
```

- `ObjectDetectionArray` 메시지 구독
- `tf2_ros.TransformBroadcaster` 초기화

1-1. tf_yolo_broadcaster.py 작성 (이어서)

```
def callback(self, msg):
    now = self.get_clock().now().to_msg()
    for i, det in enumerate(msg.detections):
        if det.class_name != 'person':
            continue

        # bbox 중심을 3D 공간에 배치 (간단 가정)
        cx, cy, w, h = det.bbox
        x = (cx - 320) / 640.0 # 정규화
        y = (cy - 240) / 480.0
        z = 1.0
```

- **person** 클래스만 필터링
- 픽셀 좌표를 정규화하여 임의의 3D 좌표로 매핑 ($z = 1\text{m}$ 고정)
- 실제 로봇에서는 깊이 칼라라(depth) 정보로 정밀 좌표 계산

1-1. tf_yolo_broadcaster.py 작성 (이어서)

```
t = TransformStamped()
t.header.stamp = now
t.header.frame_id = 'camera_link'
t.child_frame_id = f'object_{det.class_name}_{0}'
t.transform.translation.x = z
t.transform.translation.y = -x
t.transform.translation.z = -y
t.transform.rotation.w = 1.0
self.br.sendTransform(t)
self.get_logger().info(
    f'TF 발행: {t.child_frame_id} at ({z:.2f}, {-x:.2f}, {-y:.2f})')
break
```

```
def main(args=None):
    rclpy.init(args=args)
    node = TfYoloBroadcaster()
    rclpy.spin(node)
    node.destroy_node()
    rclpy.shutdown()
```

- `camera_link` 를 부모로 하여 객체 프레임 발행
- 회전 없음(`rotation.w = 1.0`)
- 첫 번째 person만 발행 (`break`)

1-2. setup.py 엔트리 포인트 등록

- `setup.py`에 새 엔트리 포인트 추가

```
entry_points={
    'console_scripts': [
        'odom_sim = tf_tutorial_pkg.odom_simulator:main',
        'tf_listener = tf_tutorial_pkg.tf_listener:main',
        'tf_yolo = tf_tutorial_pkg.tf_yolo_broadcaster:main',
    ],
},
```

- 기존 `odom_sim`, `tf_listener`와 함께 `tf_yolo` 추가

1-3. package.xml 의존성 확인

- 인터페이스 패키지 의존성 추가

```
<!-- src/tf_tutorial_pkg/package.xml -->  
<depend>tf2_ros</depend>  
<depend>geometry_msgs</depend>  
<depend>my_robot_interfaces</depend>
```

1-4. 빌드 및 실행

```
cd ~/Workspace/ros2-ws
colcon build --packages-select tf_tutorial_pkg
source install/setup.bash
```

- 실습 1에서 구축한 TF 트리를 그대로 유지한 채, YOLO 관련 노드를 추가로 실행합니다.

```
# 터미널 8: 카메라 퍼블리셔 (camera_pkg)
ros2 run camera_pkg img_pub

# 터미널 9: YOLO 퍼블리셔 (camera_pkg)
ros2 run camera_pkg yolo_pub

# 터미널 10: TF YOLO Broadcaster (tf_tutorial_pkg)
ros2 run tf_tutorial_pkg tf_yolo
```

- 웹캠 앞에 사람을 비추면 `object_person_0` 프레임이 TF 트리에 동적으로 추가됩니다.

Part 2. RViz2로 TF 시각화

2-1. RViz2 실행 및 설정

- 실시간으로 발행되는 TF 프레임과 카메라 이미지를 동시에 확인

```
rviz2
```

- Fixed Frame: `map` 으로 설정
- Add → TF: 전체 트리 확인 (`map → odom → base_link → camera_link`)
- Add → Image: Topic을 `/image_raw` 로 설정하여 원본 영상 확인
- `object_person_0` 프레임이 `camera_link` 아래에 함께 표시됨

2-2. 설정 저장

- 매번 RViz2 설정을 다시 하지 않도록 파일로 저장

```
File → Save Config As → ~/tf_tutorial.rviz
```

- 저장된 `.rviz` 파일에는 디스플레이 목록, 토픽, Fixed Frame 등 모든 설정이 포함됨
- Launch 파일에서 이 설정 파일을 불러오면 **한 번의 명령으로 동일한 시각화 환경 구성 가능**

Part 3. Launch 파일에 TF 통합

3-1. Launch 파일 작성

- 실습 1~3의 모든 노드를 하나의 Launch 파일로 통합

```
src/tf_tutorial_pkg/launch/tf_yolo_launch.py
```

```
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration
from launch.conditions import IfCondition

def generate_launch_description():
    use_rviz_arg = DeclareLaunchArgument(
        'use_rviz', default_value='false',
        description='RViz2 실행 여부'
    )
```

- `DeclareLaunchArgument`: 실행 시 `use_rviz:=true` 로 RViz2를 선택적으로 실행

3-1. Launch 파일 작성 (이어서)

```
return LaunchDescription([
    use_rviz_arg,
    # 이미지 퍼블리셔
    Node(package='camera_pkg', executable='img_pub'),
    # YOLO 퍼블리셔
    Node(package='camera_pkg', executable='yolo_pub'),
    # Static TF: map → odom
    Node(
        package='tf2_ros',
        executable='static_transform_publisher',
        arguments=['2.0', '0.0', '0.0', '0', '0', '0', 'map', 'odom']
    ),
    # Dynamic TF: odom → base_link
    Node(package='tf_tutorial_pkg', executable='odom_sim'),
    # Static TF: base_link → camera_link
    Node(
        package='tf2_ros',
        executable='static_transform_publisher',
        arguments=['0.1', '0.0', '0.2', '0', '0', '0',
                  'base_link', 'camera_link']
    ),
    # Dynamic TF: YOLO 결과
    Node(package='tf_tutorial_pkg', executable='tf_yolo'),
```

3-1. Launch 파일 작성 (이어서)

```
# TF Listener (테스트용, 필요 시 주석 처리)
Node(package='tf_tutorial_pkg', executable='tf_listener'),
# 시각화
Node(
    package='rviz2',
    executable='rviz2',
    arguments=['-d', '/home/cchyun/tf_tutorial.rviz'],
    condition=IfCondition(LaunchConfiguration('use_rviz'))
),
])
```

- `condition=IfCondition(...)`: `use_rviz` 가 `true` 일 때만 RViz2 노드 실행
- 저장된 `tf_tutorial.rviz` 설정 파일 자동 로드

3-2. setup.py에 Launch 파일 등록

- Python 패키지가 `launch/` 디렉토리의 파일을 설치하도록 설정

```
import os
from glob import glob

# ...

data_files=[
    ('share/ament_index/resource_index/packages',
     ['resource/' + package_name]),
    ('share/' + package_name, ['package.xml']),
    (os.path.join('share', package_name, 'launch'), glob('launch/*.py')),
],
```

- `data_files`에 `launch/*.py`를 추가하지 않으면 `ros2 launch` 명령어가 파일을 찾지 못함

3-3. 빌드 및 실행

```
cd ~/Workspace/ros2-ws  
colcon build --packages-select tf_tutorial_pkg  
source install/setup.bash
```

- 한 줄로 전체 시스템 실행

```
# RViz2 없이 백그라운드 실행  
ros2 launch tf_tutorial_pkg tf_yolo_launch.py  
  
# RViz2와 함께 실행  
ros2 launch tf_tutorial_pkg tf_yolo_launch.py use_rviz:=true
```

- `use_rviz:=true` 를 주면 저장된 `.rviz` 설정으로 자동 시각화

- Launch 파일 통합의 장점

- 일관성: 여러 터미널을 열지 않고 한 번의 명령으로 전체 노드 기동
- 관리 용이성: 노드 추가·제거가 코드 한 줄 변경으로 가능
- 파라미터화: `DeclareLaunchArgument` 로 실행 옵션을 외부에서 제어

- TF 트리 완성도

- `map → odom → base_link → camera_link → object_person_0`
- 정적 변환 + 동적 변환 + 객체 탐지 결과가 **하나의 트리**로 통합



- TF2 및 Launch 파일 사용 시 흔한 실수

- `my_robot_interfaces` 빌드 누락

- `tf_yolo` 노드 실행 시 인터페이스를 찾지 못해 `ImportError` 발생
- 해결: `colcon build --packages-select my_robot_interfaces` 먼저 실행

- Launch 파일 등록 누락

- `setup.py`의 `data_files`에 `launch/*.py`를 추가하지 않으면 `ros2 launch` 실패

- RViz2 설정 파일 경로 오류

- `arguments=['-d', '/path/to/tf_tutorial.rviz']` 경로가 실제 파일 위치와 다르면 RViz2가 기본 설정으로 열림